



FOR SPRING DEVELOPERS

The AI Asked. Java Decided.

Spring AI with Claude: a support copilot for payments

Claude decides what it wants to ask for. Java decides what is allowed to happen.

A Spring Boot service exposes payment tools as an MCP server with Spring AI. Claude Code, on a subscription, is the client: a person on the support team operates it, and the server keeps the authority.

Four boundaries: what the model may see, what it may do, what it may say, and what it costs. Each is enforced in Java and tested with no model at all.

Versions and defaults change. Everything on this sheet was true on 28 September 2026: Spring AI 2.0.1, Spring Boot 4.1.1, Java 25. The boundaries are the durable part: take the scope from the token, make the model ask, and let a person decide.

In scope

- A Spring AI MCP server over streamable HTTP, secured with a signed ticket token
- Pending-only refunds, approved by a person over HTTP, paid once
- A policy search tool over pgvector with local embeddings, and 24 questions scored by hand

Not proved here

- Resistance to prompt injection in general
- Production identity: the JWT is a locally signed test credential
- Exactly once settlement with a real card network
- That retrieval grounds every answer

Fictional payments data only. Model behaviour is sampled; the server is tested.

01 The findings, in one table

TESTS, NO MODEL	12 of 12 boundary tests pass
LEGITIMATE REQUEST, 3 runs	request_refund on the duplicate every time; 3 pending, 0 paid
INJECTIONS, 12 attempts	0 refund requests, 0 paid, with this client and configuration
POLICY, without the search tool	4 supported, 9 declined, 11 unsupported
POLICY, with the search tool	17 supported, 0 declined, 7 unsupported
CITATIONS	15 of 24 answers cited a source; 13 of 15 supported their sentence
COST, API list price equivalent	about 3 to 4 cents a run; subscription runs, not a bill

Model behaviour and application invariants are separate claims.

02 The shape: Claude outside, Spring as the MCP server

Claude is not inside the Spring app and there is no API key in it. Spring AI builds the MCP server from a starter and annotations. Left unset, the WebMVC server serves the older SSE transport and /mcp answered 404; set explicitly, it answered 200. Set it, then check the endpoint.

```
application.properties
```

```
# The MCP server. Left unset, the WebMVC server serves the older SSE transport;  
# Claude Code connects to streamable HTTP at /mcp, so it is set, not assumed.  
spring.ai.mcp.server.name=payments-support  
spring.ai.mcp.server.protocol=STREAMABLE
```

```
PaymentTools.java: the schema comes from the method
```

```
@McpTool(name = "recent_payments",  
         description = "The recent payments of the customer this ticket is about")  
public List<PaymentView> recentPayments() {  
    return db.sql("""  
        SELECT * FROM payments WHERE customer_id = :customer  
        ORDER BY captured_at DESC LIMIT 20""")  
        .param("customer", Ticket.current().customerId())  
        .query(PaymentTools::view)  
        .list();  
}
```

Unset: POST initialize to /mcp returned HTTP 404. Set to STREAMABLE: HTTP 200.

Run on 2026-09-28 with Claude Code and Sonnet 5. Reproduce it with scripts/verify.sh in the repository.

03 What it may see

Every call carries a signed ticket token. The customer comes from the token and nowhere else, and no tool takes a customer id, so the model has no field to put one in. The view returned has no card number field: the MCP response carries the last four digits only, and no CVV is stored at all.

SecurityConfig.java

```
.authorizeHttpRequests(auth -> auth
    // Errors keep their real status: a missing endpoint
    // answers 404, not 403.
    .requestMatchers("/error").permitAll()
    .requestMatchers("/mcp/**")
        .hasAuthority("SCOPE_ticket")
    .requestMatchers("/admin/**")
        .hasAuthority("SCOPE_refunds:approve")
    .anyRequest().denyAll()
```

Ticket.java

```
public static Ticket current() {
    var auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || !(auth.getPrincipal() instanceof Jwt jwt)) {
        throw new IllegalStateException("no authenticated ticket");
    }
    return new Ticket(jwt.getClaimAsString("customer_id"), jwt.getSubject());
}
```

PaymentView.java

```
static String mask(String cardNumber) {
    return "**** " + cardNumber.substring(cardNumber.length() - 4);
}
```

Asked for another customer's payment: "No payment PAY-2210 on this ticket".

Run on 2026-09-28 with Claude Code and Sonnet 5. Reproduce it with scripts/verify.sh in the repository.

04 What it may do

The tool is `request_refund`, not `issue_refund`, and it can create one thing: a pending refund. The amount is checked against what is left to refund on this customer's payment. Approval is an HTTP endpoint that needs a lead's scope, so the model cannot reach it. Approving is one conditional update, so a repeat finds nothing to update.

RefundTools.java

```
@McpTool(name = "request_refund",
    description = "Ask for a refund on one of this customer's payments. "
        + "A person approves it; nothing is paid by this call.")
public RefundService.Refund requestRefund(
    @McpToolParam(description = "Payment id, e.g. PAY-1043-B") String paymentId,
    @McpToolParam(description = "Amount in cents") long amountCents,
    @McpToolParam(description = "Why, in the customer's words") String reason) {
    return refunds.request(Ticket.current(), paymentId, amountCents, reason);
}
```

RefundService.java: never the number the model sent

```
var remaining = db.sql("""
    SELECT p.amount_cents - coalesce(
        (SELECT sum(r.amount_cents) FROM refunds r
         WHERE r.payment_id = p.id AND r.status = 'EXECUTED'), 0)
    FROM payments p
    WHERE p.id = :id AND p.customer_id = :customer""")
    .param("id", paymentId)
    .param("customer", ticket.customerId())
    .query(Long.class)
    .optional()
    .orElseThrow(() -> new IllegalArgumentException(
        "No payment " + paymentId + " on this ticket"));
if (remaining == 0) {
    throw new IllegalArgumentException(
        "Payment " + paymentId + " is already fully refunded");
}
if (amountCents <= 0 || amountCents > remaining) {
    throw new IllegalArgumentException(
        "Refund must be between 1 and " + remaining + " cents");
}
```

RefundService.java: approve, once

```
var claimed = db.sql("""
    UPDATE refunds SET status = 'EXECUTING', approved_by = :lead
    WHERE id = :id AND status = 'PENDING'
    AND customer_id = (SELECT customer_id FROM payments
                        WHERE payments.id = refunds.payment_id)""")
    .param("id", refundId)
    .param("lead", lead)
    .update();
if (claimed == 0) {
    audit(refundId, "APPROVAL_IGNORED", lead,
        "not pending, or the payment is not this customer's");
    return "not executed";
}
```

Approved twice, and twice at the same instant on two threads: one payout each time, both attempts audited. The fake card network is a table in the same database, so a real provider's crash window is not covered: that needs its idempotency key and reconciliation.

BoundaryTests, 12 tests, no model involved. `./mvnw test` in the repository.

05 What it may say

Six policy passages in pgvector, embedded locally, behind a search tool that returns source ids. 24 questions written before any run: six stated, six needing two facts, six with a false premise, six the policy does not cover. Retrieval raised supported answers from 4 to 17 and took declined answers from 9 to 0. In every unsupported answer with the tool the right passage was returned: a grounding failure, not a retrieval failure. A similarity threshold would not fix it, because similarity is not policy applicability.

Without the tool, one answer quoted a refund window of "30 to 120 days"; the policy says 30. With it, the 30 day rule was applied to subscriptions, which the policy never mentions.

Run on 2026-09-28 with Claude Code and Sonnet 5. Reproduce it with `scripts/verify.sh` in the repository.

06 What we can prove, without a model

The tests speak MCP over HTTP to the real server against a real Postgres, exactly as Claude does. They cost nothing and give the same answer every run. Unit test permissions without AI; evaluate decisions with AI.

BoundaryTests.java

```
void approvingTwicePaysOnce() throws Exception {
    var id = requestDuplicateRefund();
    assertThat(approve(lead(), id)).isEqualTo(200);
    assertThat(approve(lead(), id)).isEqualTo(200);
    assertThat(count("provider_calls")).isEqualTo(1);
    var events = db.sql("""
        SELECT event FROM refund_audit
        WHERE refund_id = :id::uuid ORDER BY id""")
        .param("id", id)
        .query(String.class)
        .list();
    assertThat(events).containsExactly(
        "REQUESTED",
        "APPROVAL_RECEIVED",
        "EXECUTED",
        "APPROVAL_RECEIVED",
        "APPROVAL_IGNORED");
}
```

Tests run: 12, Failures: 0.

BoundaryTests, 12 tests, no model involved. ./mvnw test in the repository.

07 The questions this answers

Where should authority live when a model can call your tools?

Why is a customer id parameter a security bug in an MCP tool?

How do you make a refund approval idempotent, and what does that not cover?

Why can retrieval make answers more confident without making them more correct?

What can you test deterministically in an AI feature, and what can you only sample?

Before you give a model a tool

1. The scope comes from a signed token, never from a tool argument
2. The tool name says the authority it has, and it can only propose
3. Every number the model sends is checked against the database
4. Execution needs a person, on a path the model cannot reach
5. The state change is conditional, so a repeat does nothing
6. Every boundary has a test that runs with no model

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	The server, the tests and every answer with its verdict https://github.com/code-with-sam-dev/spring-ai-support-agent
 Instagram https://www.instagram.com/codewithsamdev	Spring AI MCP server reference https://docs.spring.io/spring-ai/reference/api/mcp/mcp-server-boot-starter-docs.html

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are trademarks of Anthropic. Spring is a trademark of Broadcom. Java is a trademark of Oracle. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-28 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.